

COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

(c) Hamid Sarbazi-Azad

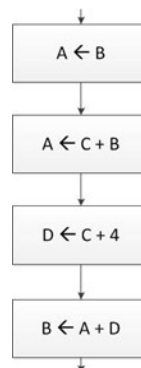
Computer Architecture -- Lecture #7: Control Unit

2

Control Unit

Control Unit is responsible for correct operation of all units and components of a digital system.

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

3

Control Unit

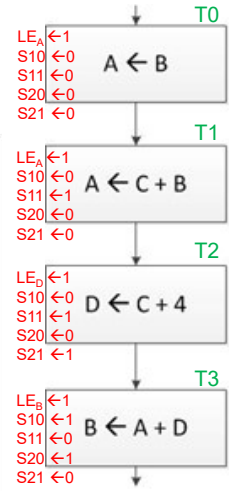
Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

We need to deign the data path first, and show all control signals on it.

→ Then, we indicate the activity of each control signal to execute each box of the flowchart.

Control signals for the data path:

- $LE_A \leftarrow 1$ (T0, T1)
- $S_{10} \leftarrow 0$ (T0, T1)
- $S_{11} \leftarrow 0$ (T0, T1)
- $S_{20} \leftarrow 0$ (T0, T1)
- $S_{21} \leftarrow 0$ (T0, T1)
- $LE_B \leftarrow 1$ (T3)
- $S_{10} \leftarrow 1$ (T3)
- $S_{11} \leftarrow 1$ (T3)
- $S_{20} \leftarrow 1$ (T3)
- $S_{21} \leftarrow 1$ (T3)



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

4

Control Unit

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

We need to deign the data path first, and show all control signals on it.

→ Then, we indicate the activity of each control signal to execute each box of the chart.

→ Now, we can write down the logic of each control signal as:

$LE_A = T0 + T1$ $LE_B = T3$ $LE_D = T2$
 $S_{10} = S_{20} = T3$ $S_{11} = T1 + T2$ $S_{21} = T2$

→ So the control unit can be designed as:

Control signals for the control unit:

- $LE_A \leftarrow 1$ (T0, T1)
- $S_{10} \leftarrow 0$ (T0, T1)
- $S_{11} \leftarrow 0$ (T0, T1)
- $S_{20} \leftarrow 0$ (T0, T1)
- $S_{21} \leftarrow 0$ (T0, T1)
- $LE_D \leftarrow 1$ (T2)
- $S_{10} \leftarrow 0$ (T2)
- $S_{11} \leftarrow 1$ (T2)
- $S_{20} \leftarrow 0$ (T2)
- $S_{21} \leftarrow 1$ (T2)
- $LE_B \leftarrow 1$ (T3)
- $S_{10} \leftarrow 1$ (T3)
- $S_{11} \leftarrow 1$ (T3)
- $S_{20} \leftarrow 1$ (T3)
- $S_{21} \leftarrow 1$ (T3)

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

5

Control Unit

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

Data Path

Control Unit

Complete System

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

6

Control Unit

Control units (or control circuits) can be categorized in:

- Hardwired control units
 - Direct Method
 - Sequencer Based Method
 - Coded State Method

Pros: Low hardware overhead, higher speed
Cons: Design inflexibility
- Microprogrammed control units
 - Pros: Design flexibility
Cons: Large hardware overhead, lower speed

Control Unit: Hardwired Control Circuits

Direct Method:

In **direct method**, we directly use the system flowchart to design its control circuit.

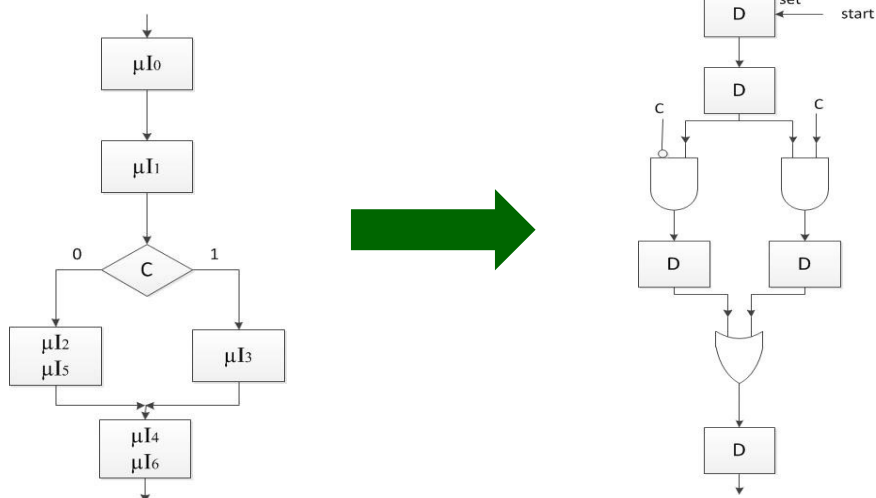
To do so:

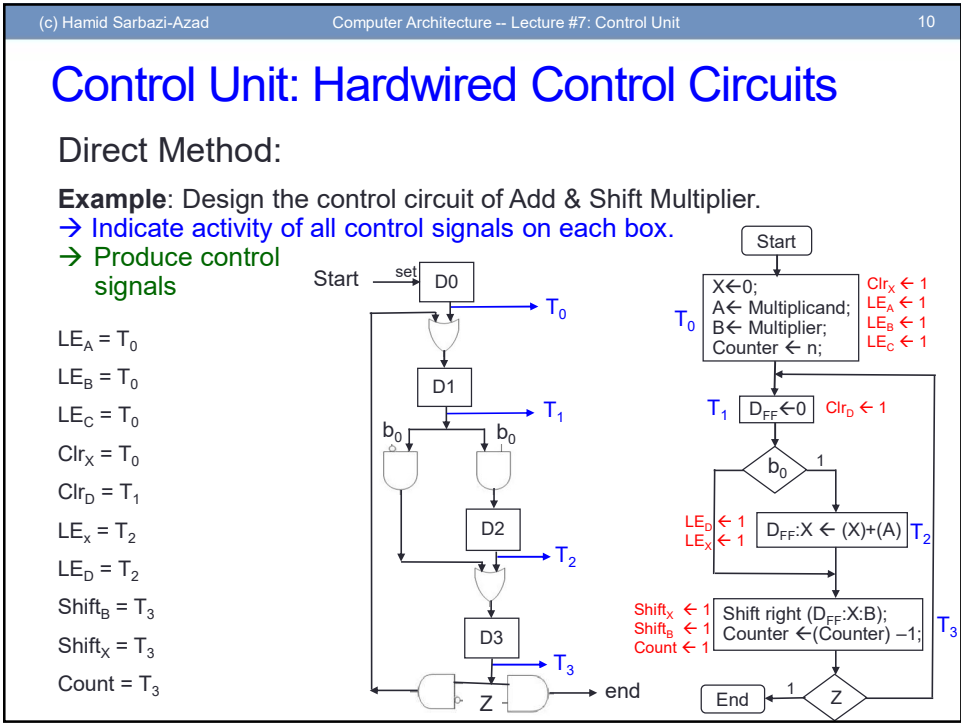
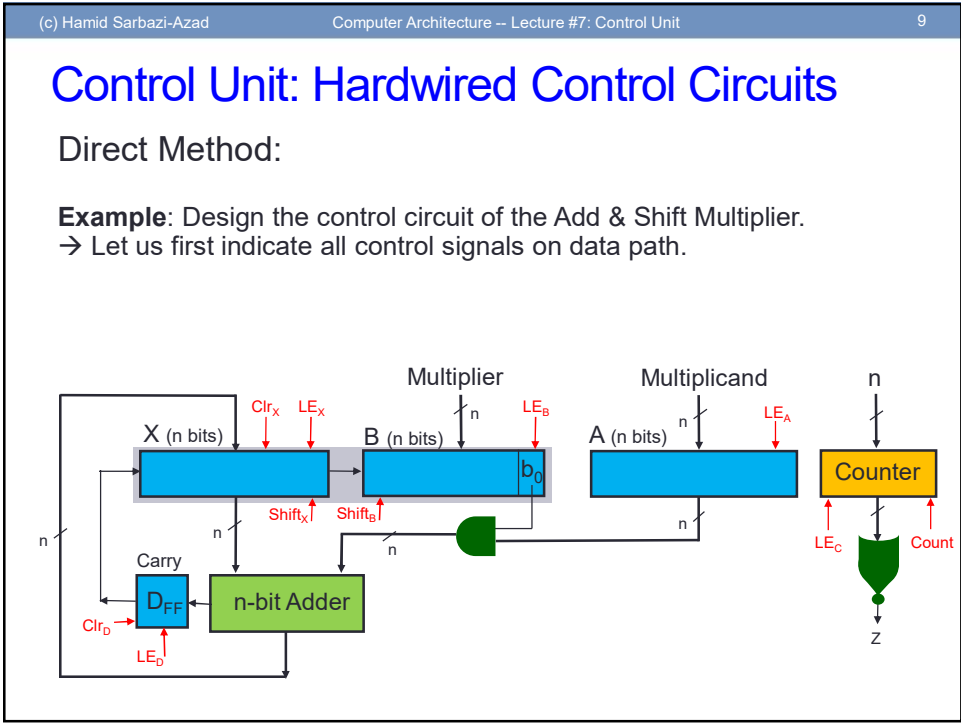
1. Each computational/transfer box (or micro-instruction box) is replaced by a D flip-flop
2. Each condition box is replaced by two AND gates
3. Junctions are replaced by OR gates

Control Unit: Hardwired Control Circuits

Direct Method:

Example: For the below chart design a control circuit.





(c) Hamid Sarbazi-Azad

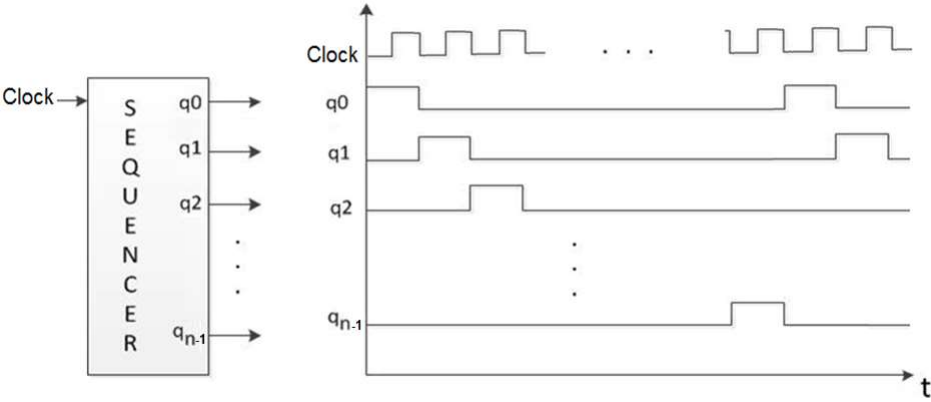
Computer Architecture -- Lecture #7: Control Unit

11

Control Unit: Hardwired Control Circuits

Sequencer Based Method:

A n-tap Sequencer is a sequential component that has n output lines to generate n consecutive pulses as shown in the figure.



(c) Hamid Sarbazi-Azad

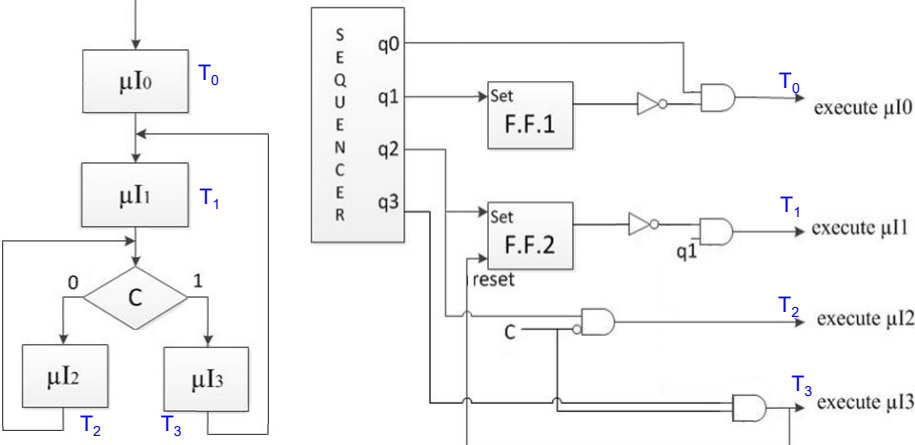
Computer Architecture -- Lecture #7: Control Unit

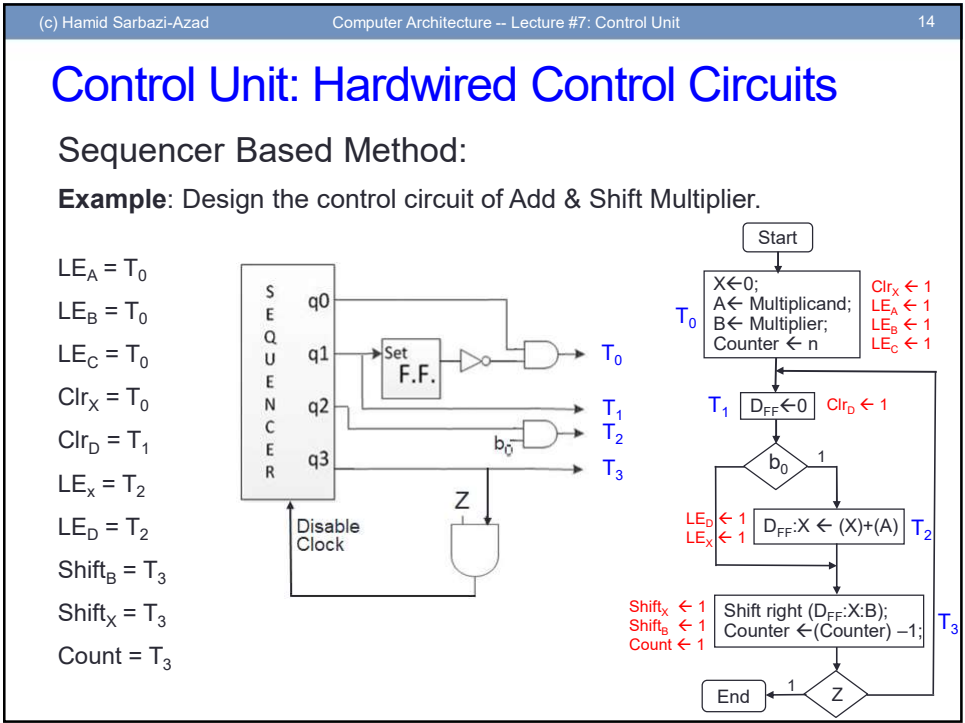
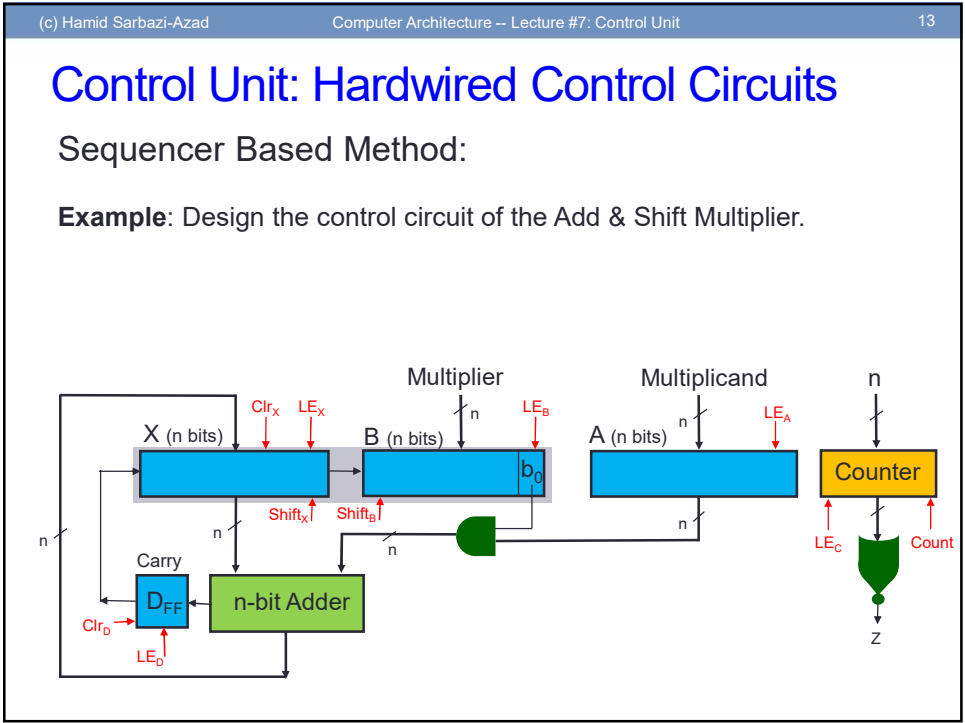
12

Control Unit: Hardwired Control Circuits

Sequencer Based Method:

Example: For the below chart design a control circuit.





(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

15

Control Unit: Hardwired Control Circuits

Coded State Method:

Each state shows a box. We need $\log_2 n$ flip-flops for n boxes.

Example: Design the control circuit of the given flowchart.

Present State	Next State			
	CS=00	CS=01	CS=10	CS=11
000	000	001	000	001
001	010	010	010	010
010	100	100	011	011
011	101	101	101	101
100	010	010	010	010
101	000	000	000	000

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

16

Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

17

Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

Present State	Next State				
	S = 0	b ₀ Z = 00	b ₀ Z = 01	b ₀ Z = 10	b ₀ Z = 11
000	000	001	001	001	001
001	000	010	010	010	010
010	000	100	100	011	011
011	000	100	100	100	100
100	000	010	101	010	101
101	000	101	101	101	101

Start 000

X ← 0;
A ← Multiplicand;
B ← Multiplier;
Counter ← n 001

D_{FF} ← 0 010

b₀ 1

011 D_{FF}:X ← (X)+(A)

Shift right (D_{FF}:X:B);
Counter ← (Counter) -1; 100

End 101

Z

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

18

Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

Present State	Next State				
	S = 0	b ₀ Z = 00	b ₀ Z = 01	b ₀ Z = 10	b ₀ Z = 11
000	000	001	001	001	001
001	000	010	010	010	010
010	000	100	100	011	011
011	000	100	100	100	100
100	000	010	101	010	101
101	000	101	101	101	101

T₀
T₁
T₂
T₃

Decoder

Next Stage Logic

D0
D1
D2

S b₀ Z

Clock

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #7: Control Unit19

Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

$LE_A = T_0$
 $LE_B = T_0$
 $LE_C = T_0$
 $Clr_X = T_0$
 $Clr_D = T_1$
 $LE_x = T_2$
 $LE_D = T_2$
 $Shift_B = T_3$
 $Shift_X = T_3$
 $Count = T_3$

```
graph LR
    T0[T0] --> Decoder
    T1[T1] --> Decoder
    T2[T2] --> Decoder
    T3[T3] --> Decoder
    Decoder --> NSL[Next Stage Logic]
    S[S] --> NSL
    b0[b0] --> NSL
    Z[Z] --> NSL
    NSL --> D0[D0]
    NSL --> D1[D1]
    NSL --> D2[D2]
    D0 --> NSL
    D1 --> NSL
    D2 --> NSL
    Clock[Clock] --> D0
    Clock --> D1
    Clock --> D2
```

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #7: Control Unit20

Control Unit: Hardwired Control Circuits

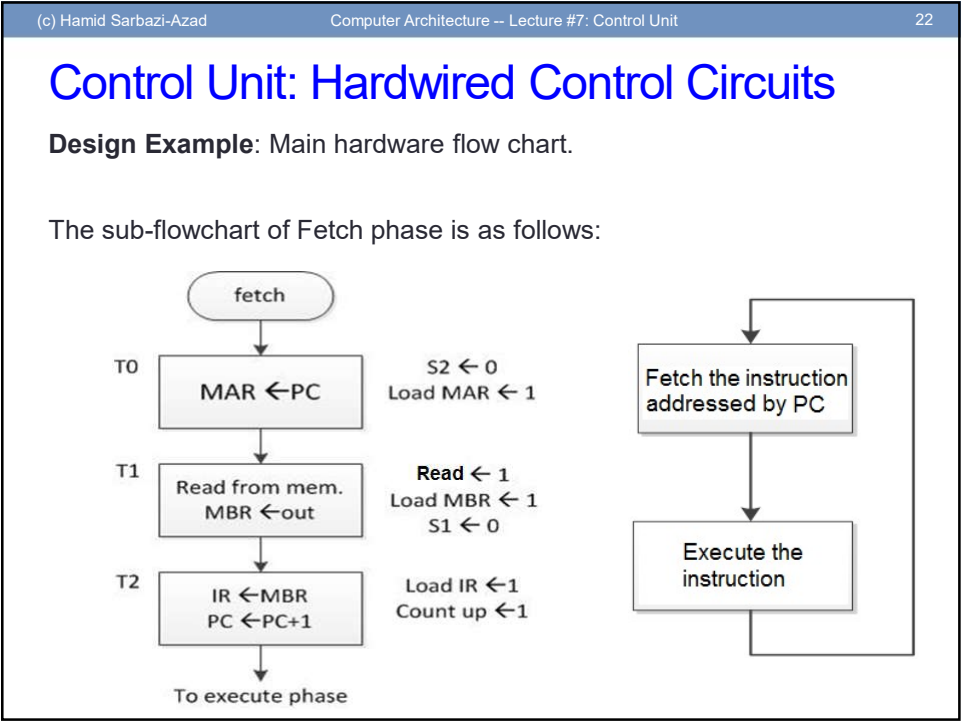
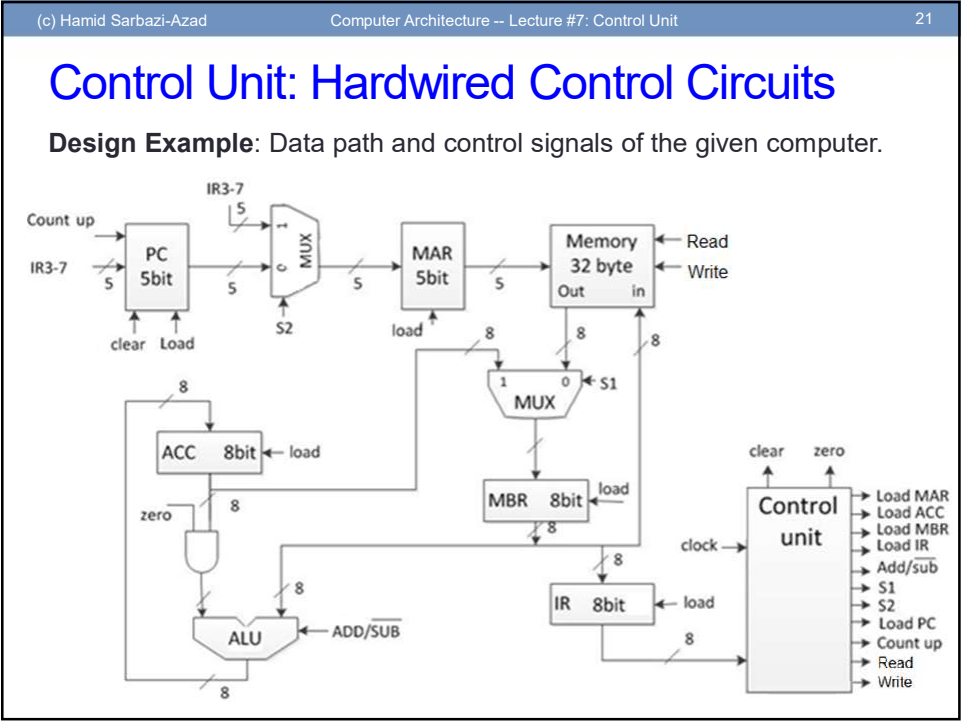
Example: Design the control unit of a simple computer defined below.

- One-address machine
- 32 bytes main memory (word = byte = addressable unit)
- Direct and implicit addressing modes
- 8-bits instructions (3-bit Opcode and 5-bit Address)

OpcodeAddress

Opcode	Mnemonic	Operation
000	LDACC	$ACC \leftarrow (M_{address});$
001	STACC	$M_{address} \leftarrow (ACC);$
010	ADD	$ACC \leftarrow (ACC) + (M_{address});$
011	SUB	$ACC \leftarrow (ACC) - (M_{address});$
100	JUMP	$PC \leftarrow Address;$
101	JACCNZ	if $(ACC) \neq 0$ then $PC \leftarrow Address;$
110	JACCNEG	if $(ACC) < 0$ then $PC \leftarrow Address;$
111	NOP	No Operation

10



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

23

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for LDACC:

T3

MAR ← addr (IR3-7)

Load MAR ← 1
S2 ← 1

T4

Read from mem.
MBR ← out

S1 ← 0
Read ← 1
Load MBR ← 1

T5

ACC ← MBR

Zero ← 0
Load ACC ← 1
Add/sub ← 1

To fetch phase

Fetch the instruction
addressed by PC

Execute the instruction

fetch

T0

MAR ← PC

S2 ← 0
Load MAR ← 1

T1

Read from mem.
MBR ← out

R/W ← 1
Load MBR ← 1
S1 ← 0

T2

IR ← MBR
PC ← PC+1

Load IR ← 1
Count up ← 1

To execute phase

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

24

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for STACC:

T3

MAR ← addr

S2 ← 1
Load MAR ← 1

T4

MBR ← ACC

S1 ← 1
Load MBR ← 1

T5

Write to memory

Write ← 1

To fetch phase

Fetch the instruction
addressed by PC

Execute the instruction

fetch

T0

MAR ← PC

S2 ← 0
Load MAR ← 1

T1

Read from mem.
MBR ← out

R/W ← 1
Load MBR ← 1
S1 ← 0

T2

IR ← MBR
PC ← PC+1

Load IR ← 1
Count up ← 1

To execute phase

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

25

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for ADD:

T3

MAR ← Addr

S2 ← 1

Load MAR ← 1

T4

Read memory
MBR ← out

Read ← 1

Load MBR ← 1

S1 ← 0

T5

ACC ← ACC+MBR

Load ACC ← 1

Zero ← 1

Add/sub ← 1

To fetch phase

Fetch the instruction
addressed by PC

Execute the
instruction

fetch

T0

MAR ← PC

S2 ← 0

Load MAR ← 1

T1

Read from mem.
MBR ← out

R/W ← 1

Load MBR ← 1

S1 ← 0

T2

IR ← MBR
PC ← PC+1

Load IR ← 1

Count up ← 1

To execute phase

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

26

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for SUB:

T3

MAR ← Addr

S2 ← 1

Load MAR ← 1

T4

Read memory
MBR ← out

Read ← 1

Load MBR ← 1

S1 ← 0

T5

ACC ← ACC-MBR

Load ACC ← 1

Zero ← 1

Add/sub ← 0

To fetch phase

Fetch the instruction
addressed by PC

Execute the
instruction

fetch

T0

MAR ← PC

S2 ← 0

Load MAR ← 1

T1

Read from mem.
MBR ← out

R/W ← 1

Load MBR ← 1

S1 ← 0

T2

IR ← MBR
PC ← PC+1

Load IR ← 1

Count up ← 1

To execute phase

13

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

27

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JACCNZ:

0

1

$\sum_{i=0}^7 ACC_i$

T3

T3

T4

T5

PC ← Addr

Load PC ← 1

To fetch phase

Fetch the instruction addressed by PC

Execute the instruction

fetch

T0

T1

T2

MAR ← PC

Read from mem. MBR ← out

IR ← MBR
PC ← PC+1

S2 ← 0
Load MAR ← 1

R/W ← 1
Load MBR ← 1
S1 ← 0

Load IR ← 1
Count up ← 1

To execute phase

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

28

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JACCNEG:

0

1

ACC₇

T3

T3

T4

T5

PC ← Addr

Load PC ← 1

To fetch phase

Fetch the instruction addressed by PC

Execute the instruction

fetch

T0

T1

T2

MAR ← PC

Read from mem. MBR ← out

IR ← MBR
PC ← PC+1

S2 ← 0
Load MAR ← 1

R/W ← 1
Load MBR ← 1
S1 ← 0

Load IR ← 1
Count up ← 1

To execute phase

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

29

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JUMP and NOP:

The diagram shows three flowcharts. The first, for NOP, shows a sequence of three empty boxes labeled T3, T4, and T5, with arrows indicating a downward flow from T3 to T4 to T5, ending with 'To fetch phase'. The second, for JUMP, shows a box at T3 labeled 'PC ← Address' with a note 'Load PC ← 1' to its right, followed by empty boxes at T4 and T5, with arrows indicating a downward flow, ending with 'To fetch phase'. The third flowchart, for the fetch phase, starts with an oval labeled 'fetch', followed by a box at T0 labeled 'MAR ← PC' with notes 'S2 ← 0' and 'Load MAR ← 1' to its right. This is followed by a box at T1 labeled 'Read from mem. MBR ← out' with notes 'R/W ← 1', 'Load MBR ← 1', and 'S1 ← 0' to its right. This is followed by a box at T2 labeled 'IR ← MBR' and 'PC ← PC+1' with notes 'Load IR ← 1' and 'Count up ← 1' to its right. Arrows indicate a downward flow from T0 to T1 to T2, ending with 'To execute phase'. Above this, a larger flowchart shows 'Fetch the instruction addressed by PC' leading to 'Execute the instruction', with a feedback loop from 'Execute the instruction' back to 'Fetch the instruction'.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

30

Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

Count up = T_2

Load PC = $JUMP.T_3 + JACCNZ.\sum_{i=0}^7 ACC_i.T_3 + JACCNEG.ACC_7.T_3$

Load MBR = $T_1 + LDACC.T_4 + STACC.T_4 + ADD.T_4 + SUB.T_4$

Load MAR = $T_0 + LDACC.T_3 + STACC.T_3 + ADD.T_3 + SUB.T_3$

Load ACC = $LDACC.T_5 + ADD.T_5 + SUB.T_5$

Load IR = T_2

$S1 = STACC.T_4$

$S2 = LDACC.T_3 + ADD.T_3 + SUB.T_3$

Read = $T1 + T4.(LDACC + ADD + SUB)$

Write = $STACC.T_5$

Zero = $ADD.T_5 + SUB.T_5$

Add/Sub = $LDACC.T_5 + ADD.T_5$

The hardware flow chart shows a 'Sequencer' block with a 'CLOCK' input and outputs for time steps T0 through T5. These outputs are connected to a 'Combinational logic' block. The combinational logic also receives inputs from a 3-bit instruction register (IR0, IR1, IR2) and a 3-bit register (D, E, C, O, D, E, R) which has outputs for LDACC (0), STACC (1), ADD (2), SUB (3), JUMP (4), JACCNZ (5), JACCNEG (6), and NOP (7). The combinational logic produces 'Control Signals' and an 'ACC' output.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

31

Control Unit: Microprogrammed CU

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

The diagram illustrates a microprogrammed control unit. The datapath includes registers A, B, C, and D, each with a load enable (LE) signal. MUX1 and MUX2 are 4-to-1 multiplexers. An adder takes two n-bit inputs and produces an n-bit output. The micro-memory contains a program with four instructions, each with a 7-bit address (LE_A, LE_B, LE_D, S10, S11, S20, S21). The micro-PC and a counter are also shown. Control signals LE_A, LE_B, LE_D, S10, S11, S20, S21 are generated from the program and used to control the datapath components.

Micro-Program:

Instruction	LE _A	LE _B	LE _D	S ₁₀	S ₁₁	S ₂₀	S ₂₁
1	1	0	0	0	0	0	0
2	1	0	0	0	1	0	0
3	0	0	1	0	1	0	1
4	0	1	0	1	0	1	0

Micro-Program:

```
T0: A ← B
T1: A ← C + B
T2: D ← C + 4
T3: B ← A + D
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

32

Control Unit: Microprogrammed CU

In order to be able to implement flowcharts with branches, some early micro-instruction formats were proposed.

Micro-instruction format

Control signals	Next μ l address

For each μ -Instruction:

- pass **Control signals** to the datapath,
- μ PC $\leftarrow$ **Next μ l address**;

The μ -architecture executing μ -programs

The diagram shows the micro-architecture executing micro-programs. The micro-PC is updated by the next micro-instruction address. The micro-IR contains control signals and the next micro-instruction address. Control signals are sent to the datapath.

Micro-Program:

```
10101010101 01010101010
10101010101 01010101010
10101010101 01010101010
...
10101010101 01010101010
10101010101 01010101010
```


(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

33

Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart (S_0 - S_{11} control signals).

μ -instruction format

Control Signals S_0 - S_{11}

12 bits

Next Addr

2 bits

μ Address	μ Instruction
	S_0 - S_3 S_4 - S_7 S_8 - S_{11} Next Addr
00	1100 0000 1001 01
01	0000 1101 0110 10
10	0011 0001 0000 11
11	0101 0010 1011 01

μ -program

The μ -architecture executing μ -programs

start

uI_0

$S_0 S_1 S_8 S_{11}$

uI_1

$S_4 S_5 S_7 S_9 S_{10}$

uI_2

$S_2 S_3 S_7$

uI_3

$S_1 S_3 S_8 S_{10} S_{11}$

clock

μPC

2 bits

μ -memory

4x14 bits ROM

1100 0000 1001 01

0000 1101 0110 10

0011 0001 0000 11

0101 0010 1011 01

μ -IR

(14 bits)

$S_0 \dots S_{11}$

NA

$S_0 - S_{11}$

(to datapath)

12 bits

2 bits

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

34

Control Unit: Microprogrammed CU

In order to be able to implement flowcharts with conditional branches, new micro-instruction formats were proposed.

Micro-instruction format

μ OPCODE

Control signals / Next μI address

For each μ Instruction:

μ Opcode = 00..0 $\rightarrow$ pass **Control signals** to the datapath;
Normal μI

μ Opcode $\neq$ 00..0 $\rightarrow$ if condition is satisfied then $\mu PC \leftarrow$ **Next μI address**;
Branch μI

The μ -architecture executing μ -programs

Load

Count

μPC

μ -Memory

1010 1010101010101010

1010 1010101010101010

1010 1010101010101010

1010 1010101010101010

μ -Program

μ -IR

μ OPCODE

Control signals / Next μI address

Logic to check if the μ Opcode is a Branch μI and condition is satisfied?

Normal μI

Control Signals (to datapath)

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

35

Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart.

μ-Instruction Format

Micro-Opcode	Control signals / Next Address
2 bits	10 bits

μ Opcode	Operation
00	Normal μl: Pass S ₀ -S ₉ to datapath
01	If c ₀ then μPC ← Next Address
10	If c ₁ then μPC ← Next Address
11	μPC ← Next Address

μ Address	μ Instruction
0000	00 1100000010
0001	00 0000110101
0010	01 0000000000
0011	00 0011000100
0100	10 0000000001
0101	00 0101001011
0110	11 0000000100

μ-program

start

Active Signals

uI₀ S₀ S₁ S₈

uI₁ S₄ S₅ S₇ S₉

c₀

uI₂ S₂ S₃ S₇

c₁

uI₃ S₁ S₃ S₆ S₈ S₉

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

36

Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart.

μ-Instruction Format

μ Opcode	Control signals / Next Address
2 bits	10 bits

μ Opc	Operation
00	Normal μl: S ₀ -S ₉ to datapath
01	If c ₀ then μPC ← Next Addr
10	If c ₁ then μPC ← Next Addr
11	μPC ← Next Address

μ Address	μ Instruction
0000	00 1100000010
0001	00 0000110101
0010	01 0000000000
0011	00 0011000100
0100	10 0000000001
0101	00 0101001011
0110	10 0000000100

μ-program

μ-Architecture

Load

Count

μPC

Clear

4 bits

μ-Memory

μ-IR

μ OPCODE

Control signals / Next μl address

10 bits

4 bits

S₀ - S₉

Normal μl

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

37

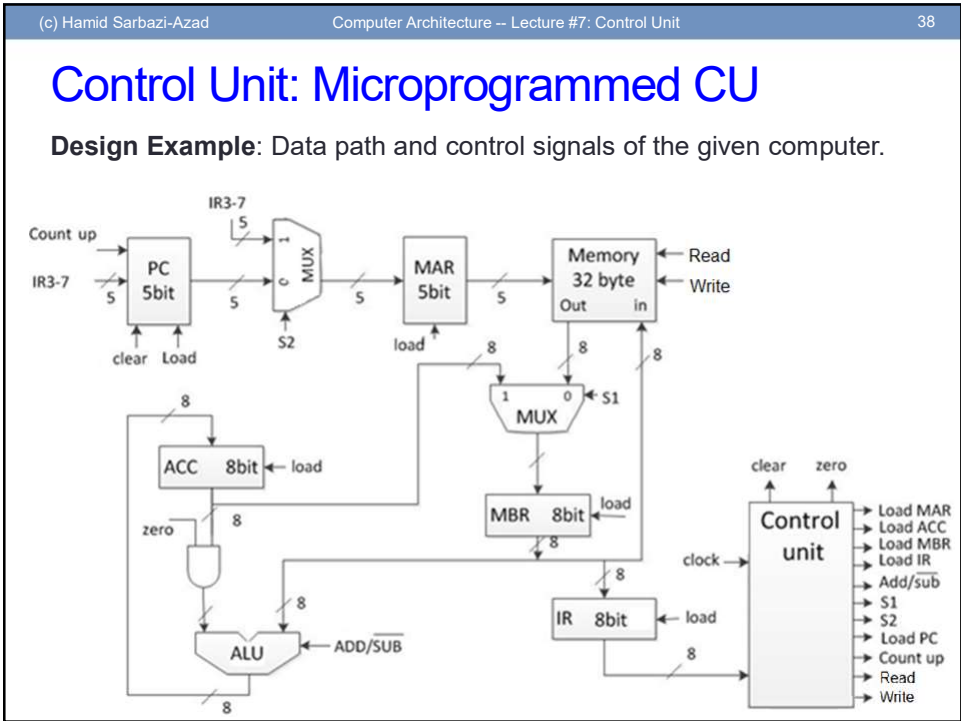
Control Unit: Microprogrammed CU

Design Example: Design the control unit of a simple computer defined below.

- One-address machine
- 32 bytes main memory (word = byte = addressable unit)
- Direct and implicit addressing modes
- 8-bits instructions (3-bit Opcode and 5-bit Address)

Instruction Format

Opcode	Address	
000	LDACC	$ACC \leftarrow (M_{address});$
001	STACC	$M_{address} \leftarrow (ACC);$
010	ADD	$ACC \leftarrow (ACC) + (M_{address});$
011	SUB	$ACC \leftarrow (ACC) - (M_{address});$
100	JUMP	$PC \leftarrow Address;$
101	JACCNZ	if $(ACC) \neq 0$ then $PC \leftarrow Address;$
110	JACCNZ	if $(ACC) < 0$ then $PC \leftarrow Address;$
111	NOP	No Operation



OPCODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
<u>OPCODE</u>	<u>Mnemonic</u>
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

OPCODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNeg
1 1 1	NOP

OPCODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

OPCODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

μ Address μ Instruction

[illegible]

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpC	Operation
000	Normal μP
001	If $(\text{ACC}) \neq 0$ then $\mu\text{PC} \leftarrow \text{Next Address}$
010	If $(\text{ACC}) < 0$ then $\mu\text{PC} \leftarrow \text{Next Address}$
011	If $\text{IR}_0 = 1$ then $\mu\text{PC} \leftarrow \text{Next Address}$
100	If $\text{IR}_1 = 1$ then $\mu\text{PC} \leftarrow \text{Next Address}$
101	If $\text{IR}_2 = 1$ then $\mu\text{PC} \leftarrow \text{Next Address}$
110	Not Used
111	$\mu\text{PC} \leftarrow \text{Next Address}$

OPCODE	Address
--------	---------

OP CODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

μ Address μ Instruction

[illegible]

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal If
001	If $(ACC) \neq 0$ then $\mu PC \leftarrow \text{Next Addr}$
010	If $(ACC) < 0$ then $\mu PC \leftarrow \text{Next Addr}$
011	If $IR_0 = 1$ then $\mu PC \leftarrow \text{Next Addr}$
100	If $IR_1 = 1$ then $\mu PC \leftarrow \text{Next Addr}$
101	If $IR_2 = 1$ then $\mu PC \leftarrow \text{Next Addr}$
110	Not Used
111	$\mu PC \leftarrow \text{Next Address}$

OPCODE	Address
--------	---------

OP CODE	Address
IR ₆ IR ₄ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNZ
1 1 1	NOP

μ Address μ Instruction

Instruction		OpC	CcC	LcC	MbR	LmAR	LACC	LrR	S ₁	S ₂	R	W	Z	A/S
FETCH	000000000	0000	0	0	0	1	0	0	1	0	1	0	0	0
	000000001	0000	0	0	0	1	0	0	0	0	1	0	0	0
	000000010	0000	1	0	0	1	0	1	0	0	0	0	0	0
	000000011	0011	0	0	0	0	0	0	0	1	0	1	1	0
	000000100	1000	0	0	0	0	0	0	0	0	1	1	0	1
	000000101	1010	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	000000110	0000	0	0	0	1	0	0	0	0	0	0	0	0
	000000111	0000	0	0	1	0	0	0	0	0	1	0	0	0
	000001000	0000	0	0	0	0	1	0	0	0	0	0	0	1
	000001001	1110	0	0	0	0	0	0	0	0	0	0	0	0
	000001010	0000	0	0	1	1	0	1	0	0	0	0	0	0
	000001011	0000	0	0	0	0	0	0	0	0	1	0	0	0
STACC	000001100	1110	0	0	0	0	0	0	0	0	0	0	0	0
	000001101	1010	0	0	0	0	0	0	0	0	0	0	0	0
	000001110	0000	0	0	0	1	0	0	0	0	0	0	0	0
	000001111	0000	0	0	1	0	0	0	0	0	0	0	0	0
	000010000	0000	0	0	0	0	1	0	0	0	0	0	0	0
	000010001	1110	0	0	0	0	0	0	0	0	0	0	1	1
ADD	000010010	0000	0	0	0	1	0	0	0	0	0	0	0	0
	000010011	0000	0	0	0	0	0	0	0	0	1	0	0	0
	000010100	0000	0	0	0	0	0	0	0	0	0	0	0	0
	000010101	1110	0	0	0	0	0	0	0	0	0	0	0	0
	000010110	1000	0	0	0	0	0	0	0	0	0	0	1	0
	000010111	1010	0	0	0	0	0	0	0	0	0	0	0	0
SUB	000010010	0000	0	0	0	1	0	0	0	0	0	0	0	0
	000010011	0000	0	0	1	0	0	0	0	0	1	0	0	0
	000010100	0000	0	0	0	0	1	0	0	0	0	0	1	0
	000010101	1110	0	0	0	0	0	0	0	0	0	0	0	0
	000010110	1000	0	0	0	0	0	0	0	0	0	0	0	0
	000010111	1010	0	0	0	0	0	0	0	0	0	0	0	0
JUMP	000011000	0000	0	1	0	0	0	0	0	0	0	0	0	0
	000011001	1110	0	0	0	0	0	0	0	0	0	0	0	0
	000011010	0010	0	0	0	0	0	0	0	1	1	0	0	0
JACCNZ	000011011	1110	0	0	0	0	0	0	0	1	1	0	0	0

μ Opcode	Control signals/Next Address
3 bits	12 bits

μ PC	Operation
000	Normal μ
001	If $(ACC) \neq 0$ then $\mu PC \leftarrow$ Next Addr
010	If $(ACC) < 0$ then $\mu PC \leftarrow$ Next Addr
011	If $IR_0 = 1$ then $\mu PC \leftarrow$ Next Addr
100	If $IR_1 = 1$ then $\mu PC \leftarrow$ Next Addr
101	If $IR_2 = 1$ then $\mu PC \leftarrow$ Next Addr
110	Not Used
111	$\mu PC \leftarrow$ Next Address

OPCODE	Address
--------	---------

OP CODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
<u>OPCODE</u>	<u>Mnemonic</u>
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

μ Address μ Instruction

[illegible]

μ Opcode	Control signals/Next Address
3 bits	12 bits

μ PC	Operation
000	Normal μ
001	If (ACC) $\neq 0$ then μ PC $\leftarrow$ Next Addr
010	If (ACC) < 0 then μ PC $\leftarrow$ Next Addr
011	If $IR_0 = 1$ then μ PC $\leftarrow$ Next Addr
100	If $IR_1 = 1$ then μ PC $\leftarrow$ Next Addr
101	If $IR_2 = 1$ then μ PC $\leftarrow$ Next Addr
110	Not Used
111	μ PC $\leftarrow$ Next Address

OPCODE	Address
--------	---------

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNZ
1 1 1	NOP

The diagram illustrates the 'New μ-architecture' with the following components and data paths:

- Inputs:** An 8-bit **Address** and a 12-bit **μPC** (labeled with Q_1, Q_0) are provided.
- μPC (Micro-Program Counter):** Receives the 8-bit address and a **Clear** signal. It outputs an 8-bit signal to the **μ-Memory** and a **Count** signal to the **MUX**.
- μ-Memory (256x15 bits ROM):** Receives the 8-bit signal from the **μPC** and outputs a 12-bit **μ-IR** (labeled **Control signals / Next μl address**).
- MUX (Multiplexer):** Receives the **Count** signal and outputs a 12-bit signal to the **μ-IR**.
- μ-IR (Micro-Instruction Register):** Receives the 12-bit signal from the **μ-Memory** and outputs an 8-bit signal back to the **μPC**.
- Control Logic:** Includes an AND gate that takes the 12-bit **μ-IR** and the 8-bit **μPC** output as inputs, and an OR gate that takes the 12-bit **μ-IR** and the 8-bit **μPC** output as inputs. The OR gate output is labeled **Normal μl**.

OPCODE	Address
IR ₀ IR ₁ IR ₂	IR ₃ IR ₄ IR ₅ IR ₆ IR ₇
<u>OPCODE</u>	<u>Mnemonic</u>
0 0 0	NOP
0 0 1	LDACC
0 1 0	STACC
0 1 1	ADD
1 0 0	SUB
1 0 1	JUMP
1 1 0	JACCNZ
1 1 1	JACCNEG

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

51

Control Unit: Microprogrammed CU

Reducing micro-memory capacity

One important issue with micro-programmed systems is the latency of micro-memory that in turn will reduce the whole CPU speed.

This is because the latency (also cost and power consumption) of micro-memory tightly depends to its size.

The smaller the micro-memory,
the better implementation under area limitation, and hence
the higher ultimate speed and lower power consumption and cost.

There are two main techniques to reduce the size of micro-memory:

1. Vertical micro-instructions
2. Nano-memory

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

52

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ -instructions

Looking at the used patterns of control signals in a micro-program, one can easily see that most bits are 0 and few bits are 1. This is because when a specific micro-instruction (like data transfer or computation boxes in the flowchart) is executed few components are involved and only the control signals associated to them are activated.

This means we can properly partition the control signals into groups where in each group at most one control signal is active in all used patterns. We can then encode each group's pattern with a short code that show the position of the 1 in the group's pattern.

If the length of a group's pattern is k bits, only $\log_2(k+1)$ bits are needed to encode the group's pattern.

→ We need some decoder (latency overhead) to decode codes and get the actual signals. ☹

Group bits positions	→	1 2 3 4 5 6 ... k
Encoded by 00...00	for	0 0 0 0 0 ... 0
Encoded by 00...01	for	1 0 0 0 0 ... 0
Encoded by 00...10	for	0 1 0 0 0 ... 0
.....		
Encoded by 11...11	for	0 0 0 0 0 ... 1

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

53

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ -instructions

Example: Consider the micro-programmed computer.

The control signals can be grouped as shown where at most one bit 1 appears in each group.

Group 1 can be coded by 3 bits and Group 2 and Group 3 each can be coded by 2 bits.

→ We will have 7 bits instead of 12 bits (~40%).

In real processors with hundreds of control signals saving can be big, say 80% 😊

C_{PC}	L_{PC}	S_1	L_{ACC}	R	W	L_{MAR}	L_{IR}	Z	S_2	$A/\bar{S}$	L_{MBR}
0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	1	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0

Group 1

Group 2

Group 3

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

54

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ -instructions

The new micro-architecture looks like:

μ -Instruction Format

μ OPCODE	Coded groups / Next μ l address
--------------	-------------------------------------

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #7: Control Unit55

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

Looking at the used patterns of control signals in a micro-program, one can easily see that different used patterns are a limited number. This is because most micro-instructions may be reused in different places of the flowchart.

This means we can store such different patterns in a special memory called **Nano-memory** and use the patterns' addressed instead of the patterns themselves.

If the number of different used patterns is k (out of theoretical 2^n possible patterns of n signals), only $\log_2 k$ bits are needed to address these k patterns in the Nano-memory.

→ A very big saving (in real systems, it can be more than 90% ☺)

→ The latency overhead of accessing the added Nano-Memory ☹

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #7: Control Unit56

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

Example: Consider the micro-programmed computer.

The control signals use only 10 different patterns.

→ We can keep them in a Nano-memory addressed by 4 bits.

That is %75 savings ☺

C _{PC}	L _{PC}	S ₁	L _{ACC}	R	W	L _{MAR}	L _{IR}	Z	S ₂	A/5	L _{MBR}
0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

57

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

The new micro-architecture looks like:

μ-Instruction Format

μ OP CODE	Nano-memory/Next μI address
-----------	-----------------------------

count_{PC},
load_{PC},
load_{MBR},
load_{MAR},
load_{ACC},
load_{IR},
S₁,
S₂,
Rd,
Wr,
Zero,
A/S

Nano-Memory

0
ACC≠0
1
ACC<0
2
IR₀
3
IR₁
4
IR₂
5
0
6
1
7

MUX

Load
Count

μPC

8 bits

Clear

μ-Memory

256x11 bits

ROM

μ-IR

μ OP CODE Nano-memory/Next μI address

4 bits

8 bits

Normal μI

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #7: Control Unit

58

Control Unit: Microprogrammed CU

Reducing micro-memory capacity:

Note that in both Vertical μ-Instruction technique and Nano-memory technique, the Control Signals field is replaced with a bit pattern whose length is the maximum of the resulted pattern length and the address length in branch micro-instructions (or number of bits addressing the μ-memory).

This is because the address bit patterns of branch μ-instructions do not comply with the philosophy behind Vertical μ-Instruction technique and Nano-memory technique. So, we must keep the address patterns unchanged.

END OF SLIDES

QUESTIONS?